B.2 Softmax Attention vs. Product-based s-norm

We now compare two fuzzy disjunction operators used to approximate the logical OR $(\max)$ over fuzzy values $\mathbf{b} = (b_1, b_2, \dots, b_m) \in (0, 1]^m$:

- Softmax Attention:

$$S_{\text{attention}}(\beta) = \sum_{i=1}^{m} \alpha_i(\beta) \cdot b_i, \quad \text{where } \alpha_i(\beta) = \frac{e^{\beta b_i}}{\sum_{j=1}^{m} e^{\beta b_j}}$$

- Product-based s-norm (disjunction):

$$S_{\text{prod}} = 1 - \prod_{i=1}^{m} (1 - b_i)$$

Both operators approximate $\max(b_1, \dots, b_m)$, but through different mechanisms. The first is a parametric softmax that converges to the maximum as $\beta \to \infty$, while the second uses De Morgan's law and the product t-norm to approximate the complement of a conjunction.

Theorem 2. *For any fuzzy values $\mathbf{b} \in (0, 1]^m$, there exists a finite $\beta^* > 0$ such that the softmax attention approximation is strictly more accurate than the product-based s-norm:*

$$\epsilon_{\textit{attention}}(\beta^*) = \max_i b_i - S_{\textit{attention}}(\beta^*) < \max_i b_i - S_{\textit{prod}} = \epsilon_{\textit{prod}}.$$

Sketch. The product-based s-norm under-approximates the disjunction, since:

$$S_{\text{prod}} = 1 - \prod_{i=1}^{m} (1 - b_i) < \max_i b_i,$$

with equality only when all but one b_i are zero. The softmax operator is a smooth approximation to $\max(b_i)$ from below and satisfies:

$$\lim_{\beta \to \infty} S_{\text{attention}}(\beta) = \max_i b_i.$$

Because $S_{\text{attention}}(\beta)$ is continuous and increasing in β and converges to the exact max, there always exists a finite $\beta^* > 0$ such that:

$$S_{\text{attention}}(\beta^*) > S_{\text{prod}} \quad \Rightarrow \quad \epsilon_{\text{attention}}(\beta^*) < \epsilon_{\text{prod}}.$$

Hence, softmax attention is strictly more accurate than the product-based disjunction for sufficiently large (but finite) β. □

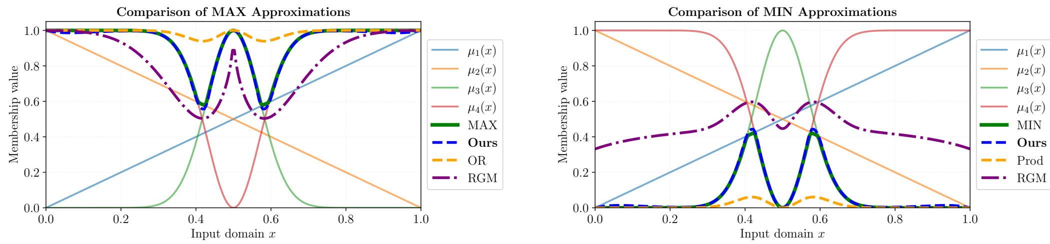
B.3 Illustrative Comparison of Conjunction and Disjunction Operators

To further illustrate the advantages of the proposed attention-based operators, we present a simple example comparing them with commonly used alternatives for approximating logical conjunction and disjunction.

We consider four fuzzy membership functions defined over the domain $x \in [0, 1]$, denoted by $\{\mu_1(x), \mu_2(x), \mu_3(x), \mu_4(x)\}$. These functions include both linear and nonlinear (Gaussian-shaped) patterns, intentionally chosen to produce diverse and challenging aggregation scenarios.

For conjunction, we compare the true minimum $\min_i \mu_i(x)$ with three differentiable approximations: the proposed attention-based softmin operator, the product t-norm $\prod_i \mu_i(x)$, and a regularized geometric mean (RGM). For disjunction, we compare the true maximum $\max_i \mu_i(x)$ with the attention-based softmax operator, the probabilistic OR, $1 - \prod_i (1 - \mu_i(x))$, and an RGM-based variant.

Figure 5 shows that the proposed attention-based operators closely track the true minimum and maximum across the entire domain. In the conjunction case, the product t-norm exhibits a strong bias toward zero whenever any input is small, resulting in overly conservative outputs and poor gradient behavior. The regularized geometric mean partially mitigates this effect but still deviates



(a) Max Operator (b) Min Operator

Figure 5: Comparison of logical operators. The proposed attention-based operators closely approximate the true MIN/MAX while remaining smooth and stable.

from the true minimum, especially in regions where inputs differ significantly. In contrast, the softmin operator adaptively concentrates weight on the smallest input, yielding a much tighter and smoother approximation.

A similar trend is observed for disjunction. The probabilistic OR operator tends to overestimate the maximum when multiple inputs take moderately large values, while the RGM-based operator introduces systematic bias due to its multiplicative structure. The proposed softmax operator, however, selectively emphasizes the largest input and provides a close and stable approximation to the true maximum.

This example highlights that the attention-based formulation achieves both higher fidelity to logical min/max behavior and improved numerical properties. In particular, it avoids the degeneracy and vanishing-gradient issues of product-based operators while preserving smoothness, making it well-suited for end-to-end differentiable rule learning.

C Syntactic Formats for Structured Rules

In ILP, structured rules must adhere to certain syntactic constraints to ensure logical validity and semantic interpretability. ANDRE incorporates these constraints during training through syntactic loss functions (see Section 3). Below, we detail the two fundamental syntactic formats enforced in this framework.

- **Head Variable Inclusion:** All variables that appear in the head of the rule must also occur at least once in the body predicates. This requirement ensures that the output of the rule is logically grounded in its input facts and that no unbound variables are introduced during inference.
  Valid example:
  $$\texttt{grandparent}(X_1, X_3)\texttt{:-}\ \texttt{parent}(X_1, X_2), \texttt{parent}(X_2, X_3).$$
  In this rule, the head predicate `grandparent` involves variables X_1 and X_3, both of which appear in the body predicates, satisfying the head variable inclusion constraint.
  Invalid example:
  $$\texttt{grandparent}(X_1, X_3)\texttt{:-}\ \texttt{parent}(X_3, X_2), \texttt{parent}(X_2, X_3).$$
  Here, variable X_1 appears in the head but is missing from the body, violating the condition and making the rule syntactically invalid.
- **Auxiliary Variable Connectivity:** Auxiliary variables—those that do not appear in the head—must either:
  1. Appear in at least two distinct body predicates, forming a connection between them, or
  2. Be completely absent from the rule (i.e., not appear in any body predicates).

  This condition ensures that auxiliary variables are semantically meaningful and contribute to the logical relationship expressed in the rule.
  Valid example (connected auxiliary):
  $$\texttt{grandparent}(X_1, X_3)\texttt{:-}\ \texttt{parent}(X_1, X_2), \texttt{parent}(X_2, X_3).$$

Variable X_2 does not appear in the head and thus is auxiliary, but it occurs in both body predicates, satisfying the connectivity constraint.

Invalid example (disconnected auxiliary):

$$\texttt{grandparent}(X_1, X_3)\,\texttt{:-}\ \texttt{parent}(X_1, X_2),\ \texttt{lives\_in}(X_4, \texttt{City}).$$

Variable X_4 appears only once and does not participate in connecting any logical entities relevant to the head predicate. Such disconnected auxiliary variables weaken interpretability and are discouraged.

These two syntactic formats are not only essential for maintaining logical consistency, but they also promote generalizability and ease of interpretation—both of which are central goals of ANDRE's framework.

D Loss Scheduling Strategy

The total training objective of ANDRE (Eq. (17)) combines semantic and syntactic losses through a set of scalar coefficients $\{\lambda_E, \lambda_S, \lambda_R, \lambda_C, \lambda_D\}$. Rather than keeping these coefficients fixed throughout training, we adopt a *progressive loss scheduling strategy* to stabilize optimization and improve rule extraction under noisy and probabilistic supervision.

Different loss components serve distinct purposes at different stages of learning. Early in training, ANDRE must prioritize semantic alignment with the data in order to identify promising predicate structures. Overly strong syntactic or discretization constraints at this stage can prematurely restrict exploration and lead to poor local optima. Conversely, in later training stages, enforcing syntactic validity and rule sparsity becomes essential for interpretability and symbolic extraction.

To address this trade-off, we gradually increase the influence of syntactic regularizers while simultaneously relaxing diversity constraints, allowing ANDRE to transition smoothly from *exploration* to *structural consolidation*.

Scheduled Loss Coefficients: Let $t \in \{0, \dots, T\}$ denote the current training epoch, and define the normalized training progress $\rho = t/T \in [0, 1]$. The loss coefficients are scheduled as follows.

Entropy Regularization (λ_E). The entropy loss (Eq. (11)) is introduced linearly to encourage early exploration over the symbolic subpredicate space:

$$\lambda_E(t) = \rho\, \lambda_E^{\max}.$$

This prevents premature convergence to arbitrary predicate selections and improves robustness in noisy settings.

Similarity Loss (λ_S). The similarity loss promotes diversity across subrules by penalizing cosine similarity between their weight vectors. Unlike the other terms, its coefficient is *decreased* during training:

$$\lambda_S(t) = \lambda_S^{\max} - \rho^2 \left(\lambda_S^{\max} - \lambda_S^{\min}\right).$$

This allows strong diversity pressure early on, while permitting convergence toward structurally similar subrules if supported by the data.

Range-Restricted Loss (λ_R). The range-restricted loss (Eq. (14)) enforces head variable inclusion. Its coefficient follows a quadratic schedule:

$$\lambda_R(t) = \frac{\rho^2}{|K_h| \cdot n}\, \lambda_R^{\max}.$$

This delayed activation allows the model to first discover semantically relevant predicates before enforcing strict variable coverage.

Connectedness Loss (λ_C). The connectedness loss (Eq. (15)) penalizes singleton auxiliary variables. To avoid early over-penalization, its weight is also increased quadratically:

$$\lambda_C(t) = \frac{\rho^2}{|K_a| \cdot n}\, \lambda_C^{\max}.$$